

Qwen3-8B REINFORCE++

Ascend NPU 算法适配与调优报告

社区任务：verl-ascend-recipe #26 | 代码交付：PR #66 | 2026 年 8 月

交付属性	内容
文档类型	算法适配与性能调优交付件
训练后端	FSDP2, BF16
Rollout 后端	vLLM + vLLM-Ascend
验证硬件	单 Ascend 910 NPU
提交脚本	reinforce_plus_plus/run_qwen3_8b_rfpp_fsdp2_npu.sh

摘要

本文记录 Qwen3-8B REINFORCE++ 在单 Ascend NPU 上的 FSDP2 适配与调优。最终 recipe 采用 BF16、actor/ref offload、动态 batch、4096 response length 和 512 MB 权重同步桶。验收训练超过 12 小时，常规吞吐为 415 至 450 tokens/s；固定验证协议下，step 0 至 step 30 的 reward mean 从 -0.7188 提升至 -0.4453，accuracy mean 从 0.1406 提升至 0.2773。运行期间未出现 NaN、inf、OOM 或 KL 发散。本文给出训练链路、健康度判据、NPU 分阶段显存诊断和最终验收结果。

目录

1	任务概述	2
2	算法与系统适配	2
2.1	训练链路与正确性约束	2
2.2	verl 与 Ascend 软件边界	4
2.3	最终 recipe	4
3	训练诊断与参数调优	5
3.1	健康度判据	5
3.2	长度与奖励	6
3.3	NPU 显存定位与加压	7
3.4	吞吐定位	9
3.5	执行流程与停止条件	9

4	验收结果	9
4.1	效果	9
4.2	性能与稳定性	10
5	复现与限制	10
5.1	复现方法	10
5.2	限制	11
6	结论	11

1 任务概述

本任务对应 verl-ascend-recipe 社区 issue #26，代码提交见 PR #66。交付范围是在 Ascend NPU 上补充 Qwen3-8B REINFORCE++ 的 FSDP2 recipe，并提供稳定性、性能和复现证据，不涉及新的算法实现。未进入正式 PR 的 Megatron 调试路径不作为验收结论。

表 1 验证环境与数据

项目	配置
模型与硬件	Qwen3-8B；单 Ascend 910 NPU，64 GB 级 HBM
数据	DAPO-Math-17k 训练集；AIME 2024 验证集
训练与推理	FSDP2、BF16；vLLM + vLLM-Ascend，TP=1
观测	console、Weights & Biases 和分阶段 npu-smi 采样
实验报告	Weights & Biases Report

验收以端到端链路完成、长跑无异常、固定协议下 checkpoint 指标改善和常规 TPS 超过 100 tokens/s 为准。由于 RL 采样存在方差，不要求相邻 step 的 reward 单调上升。

2 算法与系统适配

2.1 训练链路 with 正确性约束

REINFORCE++ 移除 critic，并对全局 batch 的 return 或 advantage 做归一化，以降低局部组归一化产生的估计偏差 [1]。对 response token t ，可将本任务中的计算过程概括为

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k,$$

(1)

$$\hat{A}_t = \frac{G_t - \mu_G}{\sigma_G + \epsilon} m_t,$$

(2)

其中 m_t 为 response mask， μ_G 和 σ_G 在有效 token 上计算。actor 以 rollout policy 产生的 response 和 old log probability 为输入，reference model 提供 reference log probability。完成 actor update 后，训练权重同步至 vLLM-Ascend，进入下一轮 rollout。

算法 1: Qwen3-8B REINFORCE++ 单 NPU 训练主循环

输入: actor 参数 θ , reference 参数 θ_{ref} , 训练集 $\mathcal{D}$, batch size $B = 256$, 每题采样数 $n = 4$

输出: 更新后的 actor、checkpoint 与验证指标

- 1 初始化 FSDP2 BF16 actor/reference; 启用参数与 optimizer offload、gradient checkpointing
- 2 初始化 vLLM-Ascend rollout engine, 设置 TP=1、KV cache 预算 0.82 与 ACL Graph
- 3 **for** $s = 0, 1, \dots, S - 1$ **do**
- 4 从 $\mathcal{D}$ 采样 prompt batch $\{x_i\}_{i=1}^B$
- 5 使用 π_θ 为每个 x_i 生成 n 条 response y_{ij} , 记录 $\log \pi_{\text{old}}(y_{ij} | x_i)$
- 6 计算 correctness score、response mask m_{ijt} 与 reference log probability $\log \pi_{\theta_{\text{ref}}}$
- 7 构造 token reward r_{ijt} ; 当前 KL coefficient = 0, KL 只记录、不进入 reward
- 8 计算 return G_{ijt} , 并在全局有效 response token 上统计 μ_G 与 σ_G
- 9 $\hat{A}_{ijt} \leftarrow (G_{ijt} - \mu_G) / (\sigma_G + \epsilon) \cdot m_{ijt}$
- 10 按不超过 8192 token 的动态 batch 重排 actor 样本
- 11 **foreach** *PPO mini batch* **do**
- 12 $\rho_{ijt} \leftarrow \exp(\log \pi_\theta - \log \pi_{\text{old}})$
- 13 计算 clipped policy objective 的 token mean, 反向传播并将 gradient norm 裁剪至 1.0
- 14 若 loss、KL 或 gradient 非有限, 则停止当前运行并保存异常上下文
- 15 使用 Adam 更新 θ
- 16 释放 rollout cache, 按 512 MB bucket 将 θ 同步至 vLLM-Ascend
- 17 记录 reward、accuracy、entropy、KL、clip fraction、长度、阶段耗时及 NPU HBM
- 18 按计划保存 checkpoint 并执行固定协议验证

算法 1 将算法更新与 Ascend 执行阶段放在同一时间轴上。rollout、actor update 和权重同步的 HBM 峰值来源不同, 因此显存参数必须按失败阶段调整。

适配后的链路必须满足以下条件:

- `critic.enable=False`, 训练流程不创建 critic 或执行 critic warmup;
- `actor_rollout_ref.rollout.calculate_log_probs=True`, 保留 old log probability 与 rollout correction 的观测入口;
- actor、reference 和 rollout 使用同一模型版本与 tokenizer 模板;
- actor 更新后完成权重同步, 下一轮 rollout 不继续使用旧权重;
- reward、advantage、loss 和 gradient norm 全部为有限值。

链路验证中, rollout 与 actor probability 的 Pearson correlation 为 0.99964, advantage mean 接近零, 说明 log probability 和全局归一化路径按预期工作。该结果仅用于排除配置错误。

最终 recipe 设置

```
algorithm.use_kl_in_reward=True
algorithm.kl_ctrl.kl_coef=0.0
actor_rollout_ref.actor.use_kl_loss=False
```

reference KL 被计算并记录, 但零系数不改变 token reward, actor 侧也没有额外 KL loss。verl 将 `kl_coef` 定义为 in-reward KL penalty 的系数 [5], 因此本文只将 KL 用于策略漂移监控。若 reward 与 entropy 同时恶化且 KL 持续升高, 可对非零 KL penalty 做独立对照; 在此之前应先排除 reward parser、输出截断和 log probability 不一致。

2.2 verl 与 Ascend 软件边界

本交付由三层软件共同完成。verl core 提供 REINFORCE++ advantage estimator、Ray trainer、FSDP2 worker、reward manager 和统一的 rollout 接口；verl-ascend-recipe 保存经过 Ascend NPU 验证的启动参数、设备环境变量和资源编排；vLLM-Ascend 作为 vLLM 的 NPU 平台插件执行 rollout。三层关系如下：

表 2 verl-ascend-recipe 软件职责边界

组件	本任务中的职责
verl core	执行 <code>verl.trainer.main_ppo</code> ，计算 reward、return/advantage、old/ref log probability 和 actor loss，管理 FSDP2 worker 与 checkpoint。
verl-ascend-recipe	提供 Qwen3-8B 单 NPU shell recipe，固化 Ascend 设备可见性、FSDP2/offload、vLLM-Ascend 并发、ACL Graph 与权重同步参数。
vLLM-Ascend	将 vLLM rollout engine 映射到 Ascend NPU，管理模型执行、KV cache、ACL Graph capture/replay 和生成调度。

脚本使用 `rollout.name=vllm`，设备分发由 vLLM-Ascend 插件完成。verl 沿用上游 vLLM 的 `memory utilization` 和 `graph` 字段：前者控制 NPU HBM 中的 KV cache 预算，后者映射为 ACL Graph 配置。字段名中的 GPU/CUDA 不表示训练运行在 GPU 上。

单 NPU recipe 还显式设置以下 Ascend 运行约束：

```
ASCEND_RT_VISIBLE_DEVICES=0
ASCEND_VISIBLE_DEVICES=0
HCCL_CONNECT_TIMEOUT=5400
USE_OPTIMIZED_MODEL=0
trainer.device=npu
```

`USE_OPTIMIZED_MODEL=0` 避免纯推理优化路径干扰 RL 场景的 log probability 和在线权重更新。当前 TP=1，HCCL 不作为吞吐优化来源。

2.3 最终 recipe

表 3 最终训练配置

配置域	关键参数	取值
算法	Advantage / critic	<code>reinforce_plus_plus</code> ; critic disabled
算法	Optimizer / loss	learning rate 1×10^{-6} ; gradient clip 1.0; token-mean loss
Batch	全局与更新批量	train batch 256; PPO mini batch 16; rollout $n = 4$
序列	长度上限	prompt 1024; response 4096; model length 5120

配置域	关键参数	取值
采样	解码参数	temperature 1.0; top-p 1.0; top-k -1; enable_thinking=False
FSDP2	精度与节省显存	BF16; actor/ref parameter offload; optimizer offload; gradient checkpointing
动态 batch	生效阶段 / token cap	actor、rollout/ref log probability; 8192 token
Rollout	HBM 与并发	utilization 0.82; 24 agent workers; 32 max sequences; 16384 max batched tokens
Rollout	缓存与图模式	chunked prefill; prefix caching; free cache engine; ACL Graph FULL_DECODE_ONLY
权重同步	Bucket	512 MB
Reward	Manager / shaping	DAPO reward manager; overlong shaping disabled; penalty factor 0
验证	协议	与训练采样分离; 按固定 checkpoint 执行; 使用完整测试文件

verl 建议在 dynamic batch 下以每设备最大 token 数控制 forward/backward 负载，并允许 forward-only 的 log probability token cap 高于 actor update cap[3]。本配置的 8192 token cap 为最大 prompt 与 response 长度之和的约 1.6 倍。显存仍有余量时，应通过单变量实验提高 actor token cap，不同时增大 batch、response length 和 rollout 并发。

在 Ascend NPU 上，actor_rollout_ref.actor.use_torch_compile=False 和对应 FSDP2 compile 开关保持关闭。历史调试曾出现 forked subprocess 重新初始化 NPU 失败，关闭相关路径后未再复现。ACL Graph 仅用于 vLLM-Ascend rollout。

3 训练诊断与参数调优

3.1 健康度判据

单个 step 的 reward 回落不等价于训练异常。健康度判断同时覆盖正确性、优化动态、输出分布和系统性能：

1. 硬失败：出现 OOM、NaN、inf、worker 重启、权重同步失败或 checkpoint 损坏，立即停止并定位；
2. 稳定性异常：KL、gradient norm、clip fraction、entropy 或截断率持续偏离近期基线，需要回退最近一次参数修改；
3. 效果异常：训练 reward 与固定协议下的 validation accuracy 在多个 checkpoint 上同时恶化，才判断训练目标可能失效。

表 4 训练健康度指标与调节动作

指标	判定信号	调节动作
Reward 与验证指标	观察 5 至 10 step 滑动均值及 checkpoint 趋势。相同协议下 reward 与 accuracy 同时恶化才构成效果异常。	先检查 reward parser、答案格式、shaping 和评测协议，再调整学习率。
Entropy 与输出分布	快速下降并伴随输出同质化或 accuracy 停滞，提示探索塌缩 [2]。	联合检查 temperature、clip fraction、reward 分布和 response length，必要时降低更新强度。
KL 与 clip fraction	actor/ppo_kl、rollout_corr/kl 应有限且围绕近期基线变化。KL 突增或 clip fraction 长期过高表示更新或权重同步异常。	核查 old log probability、权重同步和 dtype，再调整 learning rate、clip ratio 或 PPO epochs。
Gradient norm	裁剪后应为有限值，不应持续跨数量级跳变。	非有限值时停止训练；持续尖峰时降低 learning rate 或 token batch，并检查异常 reward。
Response length	平均长度、上限命中率和 aborted ratio 应稳定。上限命中率持续超过 10% 时进入告警，超过 30% 时优先处理。	检查 thinking 模式、EOS、答案格式和长度上限，避免用不合理的长度惩罚强行压短。
HBM、TPS 与阶段耗时	按 rollout、actor update、weight sync 和 checkpoint 分别观测。提高 HBM 后 TPS 连续下降超过 5% 时回退。	根据 OOM 阶段和 timing_s/* 调整对应参数，同时检查 offload 引入的主机内存与传输开销。

表 4 中的数值告警线只适用于当前单 NPU recipe。

DAPO 指出，entropy collapse、截断样本的 reward noise 和不合理的 overlong shaping 会破坏长推理 RL 的稳定性 [2]。verl 也将 policy loss、KL、entropy、gradient norm、clip fraction、score、reward 和长度指标定义为不同观测量 [4]。reward 可能含 KL 或长度 shaping，不能与原始 correctness score 混用。

3.2 长度与奖励

早期实验保留 Qwen3 thinking，并将 response length 设置为 8192 或 10240。实测上限命中率达到 68.75% 至 78.13%，generation 占据大部分 step 时间。将上限继续从 8192 提高至 10240 后，截断率虽有下降，但 reward 未改善，单步吞吐降至约 82 tokens/s。该结果说明瓶颈不是单纯的长度上限，而是输出模式、token 数和 actor backward 峰值共同作用。

最终关闭显式 thinking block，并将 response length 收敛至 4096。稳定配置下平均 response length 为 1.5k 至 1.8k token，早期 22 step 的平均截断率约为 4.08%，aborted ratio 为零。只有正确样本因达到上限而丢失答案，且上限命中率持续偏高时才应增加 response length。增加长度后须重新测量 generation 时间、actor backward HBM、答案完整率和最终精度。

早期配置令 `overlong_buffer_len` 与最大 `response length` 同为 4096，导致验证样本进入惩罚区间。此时 `accuracy` 略升而合成 `reward` 下降，不能据此判断策略退化。最终 `recipe` 关闭 `overlong shaping`，并将 `penalty factor` 设为零。若重新启用，应同时记录原始 `score`、长度 `shaping`、总 `reward`、`response length` 和 `accuracy`。

数学 `prompt` 超过 1024 token 时不宜左截断，因为题目条件可能被删除。正式数据准备应过滤或压缩超长 `prompt`，并记录处理数量。

3.3 NPU 显存定位与加压

混合训练的显存峰值随阶段变化。`gpu_memory_utilization` 主要控制 vLLM KV cache 预留，不直接约束 `actor backward`、权重同步或 `checkpoint` 峰值。vLLM-Ascend 文档指出，KV cache 接近耗尽时会触发 `preemption` 和重计算，可通过增加可用 KV cache，或降低 `max_num_seqs` 与 `max_num_batched_tokens` 缓解 [6]。

表 5 训练阶段、主要显存来源与首选参数

阶段	主要显存来源	首选调节参数
Rollout	模型权重、KV cache、ACL Graph capture、并发序列	<code>gpu_memory_utilization</code> 、 <code>max_num_seqs</code> 、 <code>max_num_batched_tokens</code> 、 <code>model length</code>
Log probability	<code>actor/ref forward activation</code> 与动态 token batch	<code>log probability token cap</code> 、 <code>dynamic batch</code> 、 <code>ref offload</code>
Actor update	<code>activation</code> 、 <code>gradient</code> 、 <code>optimizer state</code>	<code>actor token cap</code> 、 <code>PPO mini batch</code> 、 <code>response length</code> 、 <code>gradient checkpointing</code> 、 <code>optimizer offload</code>
Weight sync	同步 <code>buffer</code> 与 rollout engine 临时内存	<code>update weight bucket</code> 、 <code>free cache engine</code> 、 <code>parameter offload</code>
Checkpoint	参数和 <code>optimizer state</code> 的 CPU materialization	保存频率、保留数量、主机内存、 <code>optimizer state</code> 策略

10240 `response length` 实验中，将 `rollout memory utilization` 从 0.90 依次降至 0.86、0.82 和 0.78 后，仍在 `actor backward` 阶段 OOM。典型失败点已分配约 57.9 GB，仅剩约 1.2 GB，而 `backward` 仍需约 5.65 GB。这说明 `actor token batch` 是主导峰值，继续压低 KV cache 预算不能解决问题。最终通过缩短 `response`、采用 `FSDP2`、开启 `offload` 和 `dynamic batch` 解决。

另一组实验关闭 `actor offload` 后，在 `Adam state` 初始化或 `actor update` 阶段尝试额外分配约 1.16 GB 时 OOM。恢复 `actor parameter/optimizer offload` 后训练可连续运行。早期权重同步还出现过额外申请约 370 MB 失败，采用 512 MB 分桶并配合 `FSDP2/offload` 后不再复现。

稳定配置中，`rollout` 阶段 vLLM worker 约占 39.9 至 40.7 GB，整卡 HBM 约为 46.8 至 47.1 GB；`actor update` 阶段 vLLM worker 释放至约 88 MB，整卡 HBM 约为 16.9 至 17.1 GB。最终验收 `step 30` 的 `actor process` 记录 `allocated` 12.40 GB、`reserved` 18.06 GB。进程指标只用于同阶段回归，不能代表整卡峰值。

显存加压按以下顺序执行，每轮只修改一个变量：

- 1. 固定模型长度、train batch、rollout n 和 offload，先得到无 OOM 的完整 step；
- 2. 以 0.02 至 0.05 的步长提高 `gpu_memory_utilization`，观察 KV cache usage、preemption、generation TPS 和整卡 HBM；
- 3. KV cache 尚有余量时提高 `max_num_seqs`，随后提高 `max_num_batched_tokens`；
- 4. rollout 达到稳定平台后，再提高 actor 与 log probability token cap；
- 5. 最后评估减少 offload。单卡 Qwen3-8B 的实测结果不支持关闭 actor offload，因此该项不进入默认 recipe。

各阶段不必达到相同 HBM 百分比。rollout 需要为 ACL Graph capture、权重同步和临时分配保留空间。HBM 增加而 TPS 无改善，或出现 preemption、step time 长尾和 worker restart 时，应回退上一档。过大的 batched token 范围会增加 graph、padding 或 stream 压力，不保证提高性能 [7]。

verl 日志中的 `actor/perf/max_memory_allocated_gb` 只统计对应 PyTorch NPU 进程，不能覆盖独立的 vLLM worker、ACL Graph、运行时和 HCCL 内存。显存调优必须同时采集 trainer 指标与 `npu-smi`。建议在每个完整 step 内至少覆盖 rollout、actor update、weight sync 和 checkpoint 四个时间点：

```
npu-smi info
npu-smi info -l
```

采样记录应包含时间、step、阶段、整卡 HBM、进程 HBM 和 AICore utilization。VLLMWorker 对应 rollout，`ray::WorkerDict` 通常对应 actor/ref worker。只有相同阶段的数据可用于显存回归。

表 6 Ascend NPU 观测信号与判断

观测信号	可能原因	处理
Rollout HBM 与 AICore 同时升高，TPS 上升	KV cache 和并发有效利用 NPU	继续小步增加并发，直到吞吐平台或出现 preemption。
Rollout HBM 高但 AICore 与 TPS 不升	调度、padding 或 graph bucket 不匹配	检查 max sequences、batched tokens、CPU affinity 和 capture sizes。
Actor update OOM	activation、gradient 或 optimizer state 峰值	降低 actor token cap、mini batch 或 response length，启用 checkpointing/offload。
Weight sync 瞬时 OOM	同步 bucket 与 rollout runtime 临时内存叠加	减小 update bucket，确认 cache engine 已释放 KV cache。
HBM 正常但 actor update 慢	Offload、CPU fallback 或 算子瓶颈	检查 CPU memory、NUMA、fallback warning 和阶段耗时。
HBM 同阶段持续增长	cache 未释放、graph 重复捕获或资源泄漏	检查 worker 重启、graph capture 和 checkpoint 后资源状态。

vLLM-Ascend 提供以下 allocator 配置，用于显存紧张时减少分配碎片 [8]：

```
PYTORCH_NPU_ALLOC_CONF=expandable_segments:True
```

该选项只在出现碎片化证据后单独验证，不写入默认 recipe。若 OOM 请求量明显大于空闲 HBM，仍应调整 token batch、长度或 offload。

3.4 吞吐定位

generation 和 actor update 是主要耗时项。较早的稳定 step 中，两者分别约为 470 和 352 秒，old/reference log probability 约为 34 和 32 秒。因此调优优先级是 rollout 并发和 actor token batch，而不是 reward worker 数量。

最终 recipe 使用 0.82 rollout memory utilization、24 agent workers、32 max sequences 和 16384 max batched tokens。性能结果按常规 step 与 validation/checkpoint step 分组，避免将周期任务误判为训练回退。

3.5 执行流程与停止条件

复用 recipe 时依次进行配置校验、完整单步、十步稳定性、显存加压和长跑验收。配置校验固定模型、tokenizer、chat template、数据字段、reward parser 和软件版本；完整单步要求 rollout 至权重同步闭环且所有数值有限；十步稳定性检查 KL、gradient norm、response length、clip fraction 和 worker 状态；显存加压按第 3.3 节执行；长跑固定参数，并在 checkpoint 前后检查 HBM、主机内存和恢复能力。

出现 NaN/inf、同阶段重复 OOM、worker 重启、持续 KV preemption、权重同步失败，或 KL 与 gradient norm 跨数量级异常时，应立即停止并回退。若 HBM 上升而 TPS 持续下降，或 reward 上升但 accuracy、答案格式和输出多样性同时恶化，也不继续扩大配置。

4 验收结果

4.1 效果

表 7 使用相同的 AIME 2024 验证协议和采样参数，用于比较 checkpoint 趋势。

表 7 固定验证协议下的 checkpoint 结果

Step	Reward mean	Reward best	Acc. mean	Pass	Majority
0	-0.7188	-0.2986	0.1406	0.3507	0.2249
10	-0.7266	-0.2625	0.1367	0.3688	0.2234
30	-0.4453	0.1063	0.2773	0.5531	0.3763

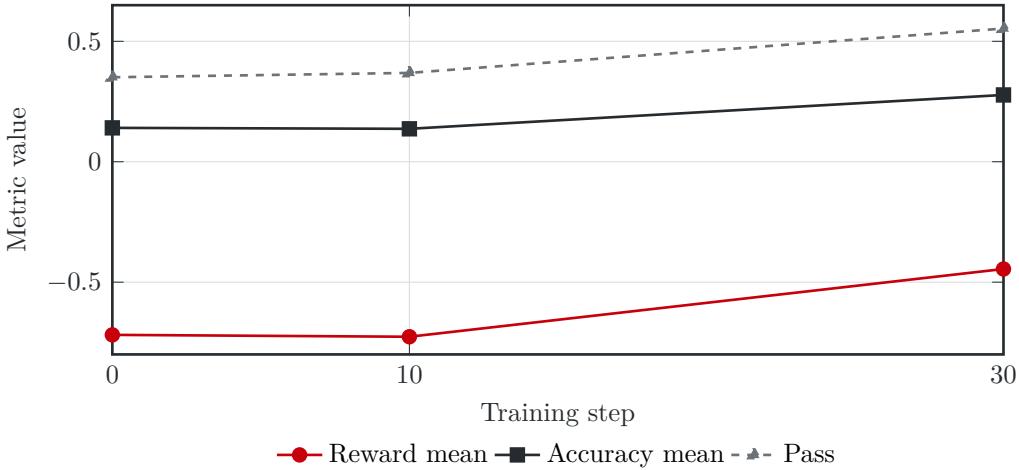


图 1 Reward 与 AIME 2024 accuracy 的 checkpoint 趋势。step 10 有短期回落，主要改善出现在 step 10 至 step 30。

step 0 至 step 30, reward mean 提高 0.2735, reward best 提高 0.4049 并由负转正; accuracy mean、pass 和 majority 分别提高 0.1367、0.2024 和 0.1514。step 10 出现短期回落，step 30 的多项指标同步改善构成训练有效的主要证据。

4.2 性能与稳定性

表 8 单 NPU 吞吐结果

场景	Throughput
常规训练 step	约 415 至 450 tokens/s
Step 1	445.51 tokens/s
Step 10, 含 validation/checkpoint	376.32 tokens/s
Step 30, 含 validation/checkpoint	385.91 tokens/s

验收长跑超过 12 小时，期间未出现 NaN、inf、OOM、RuntimeError 或 KL 发散。step 30 的 rollout_corr/kl 为 3.42×10^{-4} ，actor/ppo_kl 为 2.85×10^{-5} 。另一组单 NPU 运行生成了 global_step_100 checkpoint，两组结果不合并为同一条实验轨迹。当前结果满足运行时长、reward 趋势和绝对 TPS 要求。由于缺少同版本 A100 对照，本文不报告跨硬件性能倍率。

5 复现与限制

5.1 复现方法

数据目录包含 train.parquet 与 test.parquet。在 verl 根目录执行：

```
bash /path/to/verl-ascend-recipe/reinforce_plus_plus/\
run_qwen3_8b_rfpp_fsdp2_npu.sh \
  --data_path=/path/to/data \
  --model_path=/path/to/Qwen3-8B
```

脚本支持将未识别的命令行参数继续传给 `verl.trainer.main_ppo`，可使用 Hydra override 做单变量实验。例如：

```
bash run_qwen3_8b_rfpp_fsdp2_npu.sh \  
  --data_path=/path/to/data \  
  --model_path=/path/to/Qwen3-8B \  
  actor_rollout_ref.rollout.gpu_memory_utilization=0.80 \  
  actor_rollout_ref.rollout.max_num_seqs=24
```

每次运行应保存 git commit、完整 Hydra 配置、软件版本、训练日志、W&B run、checkpoint、validation 输出、npu-smi 分阶段采样和异常堆栈。

5.2 限制

本次 PR 只验证 FSDP2 recipe，未确定非零 KL penalty 的最优系数。Offload 是单 NPU 稳定运行的必要条件，但会增加主机内存占用和传输开销。`use_remove_padding=True` 在实测软件栈中出现 jagged-to-padded-dense CPU fallback，可能影响 actor update。历史日志未保留完整的 CANN、torch、torch-npu、vLLM-Ascend 和 verl 版本矩阵，后续复现应通过 `REQUIRED_VERL.txt`、容器标签和环境清单补齐。当前结果没有同版本 A100 基线，因此只报告绝对 TPS 和固定协议下的趋势。

6 结论

适配中的主要问题是长序列引起的 actor activation 峰值，以及 rollout、actor update 和权重同步在同一 NPU 上形成的阶段性资源竞争。最终通过 4096 response length、FSDP2、actor/ref offload、动态 token batch 和权重同步分桶稳定训练，再逐级提高 rollout 并发，将常规吞吐提升至 415 至 450 tokens/s。

验收同时检查链路正确性、KL 与 gradient、输出长度、固定协议精度、阶段吞吐和长跑稳定性。该方法区分算法异常与系统瓶颈，适用于 verl-ascend-recipe 中同类 critic-free RL 任务。

参考文献

- [1] Jian Hu, Jason Klein Liu, Haotian Xu, and Wei Shen. REINFORCE++: Stabilizing Critic-Free Policy Optimization with Global Advantage Normalization. arXiv:2501.03262, 2025. <https://arxiv.org/abs/2501.03262>.
- [2] Qiying Yu et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. Advances in Neural Information Processing Systems, 2025. https://papers.nips.cc/paper_files/paper/2025/file/a4277440d50f1f15d2cb4c14f7e0c0d2-Paper-Conference.pdf.
- [3] verl contributors. Performance Tuning Guide. https://verl.readthedocs.io/en/latest/perf/perf_tuning.html.
- [4] verl contributors. 训练配置参数与指标说明. https://verl.readthedocs.io/en/latest/ascend_tutorial/dev_guide/model_dev/parameter_and_metrics.html.

-
- [5] verl contributors. Config Explanation. <https://verl.readthedocs.io/en/latest/examples/config.html>.
 - [6] vLLM-Ascend contributors. Frequently Asked Questions. <https://docs.vllm.ai/projects/ascend/en/latest/faqs.html>.
 - [7] vLLM-Ascend contributors. ACL Graph. https://docs.vllm.ai/projects/ascend/en/latest/developer_guide/feature_guide/ACL_Graph.html.
 - [8] vLLM-Ascend contributors. Optimization and Tuning. https://docs.vllm.ai/projects/ascend/en/latest/developer_guide/performance_and_debug/optimization_and_tuning.html.